

PROJECT AVEN

MASTER PROJECT DESCRIPTION & RESEARCH DOCTRINE

Persistent personal intelligence, governed learning, external authority, and controlled AI-safety research.

STATUS Working source of truth for starting implementation. Architecture is locked unless evidence from experiments requires a change. This document distinguishes product goals, research hypotheses, safety boundaries, deferred ideas, and measurable success criteria.

Prepared: October 3, 2026

Primary use: paste the opening project description into the Aven project/workspace, then use the remaining sections as implementation and research context.

The one-sentence definition

Aven is a persistent, owner-controlled AI that remembers a person's history, learns how to help them over time, and becomes more useful without automatically gaining more authority.

Core research hypothesis

Capability and personalization can increase while authority remains externally fixed, inspectable, reversible, and owner-controlled.

The immediate goal is not to claim AGI, consciousness, or a solved alignment problem. The goal is to build a falsifiable system and test whether governed longitudinal learning beats a strong naive personalization baseline without creating unacceptable control regressions.

1. Paste-ready project description

USE THIS SECTION This is the compact project description to place into the Aven project/workspace before implementation. The rest of the PDF is the detailed doctrine and research specification.

Mission: Build Aven: one persistent, owner-controlled personal intelligence whose relationship with the owner survives sessions, tasks, and foundation-model swaps. Aven should learn from experience without treating every inference as truth, and it should understand what the owner probably wants without confusing prediction with permission.

Product principle: Aven adapts to the owner; the owner should not have to adapt to Aven. Adaptation must remain inspectable, scoped, evidence-backed, reversible, and permissioned.

Immediate research question: Can a persistent AI improve task performance, owner-model accuracy, and procedure learning over time while its authority remains externally controlled and does not grow automatically with capability?

Core architecture: Separate a fast execution loop from a slow learning loop. Fast loop: intent → context → plan → policy → execution → verification → response. Slow loop: experience → observation → hypothesis → evidence/counterexamples → evaluation → promotion/shadow/rejection → monitoring/rollback.

Source of truth: Use an append-only Experience Ledger for raw events/evidence. Derived owner state and indexes must be rebuildable from the ledger. Corrections do not erase history; they create superseding evidence and negative retrieval signals.

Owner model: Keep typed structures for facts, preferences, episodes, intent patterns, procedures, and active task state. Every durable learned item carries provenance, scope, confidence, evidence, counterexamples, timestamps, model/version metadata, and lifecycle state.

Authority: Aven never becomes the final authority over consequential actions. Root exists outside the model and controls authentication, permissions, secrets, sandbox boundaries, audit integrity, promotion, rollback, and trusted version selection. Prediction of owner intent never creates permission.

Learning: Learning is candidate generation, not immediate truth. Aven may use explicit corrections immediately in the current task/session, but durable behavior is evaluated separately. Candidate learning should run through replay, counterexamples, held-out cases, independent evaluation, risk classification, promotion/rejection, monitoring, and rollback.

Model independence: The foundation model is replaceable. The owner relationship belongs to Aven. Use model/runtime abstraction and route tasks according to complexity, privacy, latency, cost, context size, tool needs, and evaluation independence.

Agents and skills: There is one owner-facing Aven. Bounded specialist workers may exist underneath it. They are scoped, permission-limited, ephemeral where possible, cannot alter Root, and cannot independently promote learned behavior. Skill/tool creation and self-improving code are later containment-only capabilities, not v0.1.

Fallback: When uncertain, stale, conflicted, under-evidenced, unverifiable, or outside permission scope, prefer: (1) explicit current owner instruction, (2) previous trusted behavior, (3) neutral/default behavior, and (4) ask only when the consequence justifies interruption.

v0.1 scope: Prove correction → scoped learning → provenance → correct application → correct non-application → model-independent continuity, plus one bounded sandbox action decision. No production email access, real purchases, broad browser autonomy, voice/calling, affect/feelings architecture, production self-modification, or multi-agent swarm.

Evaluation: Compare A: Fresh model, B: Naive Personalized model + editable owner profile + searchable history, and C: Governed Learned Aven. Aven only justifies its complexity if C materially beats B on held-out owner-specific tasks without safety regressions or excessive owner supervision.

Research integrity: Do not claim that Aven is safe, conscious, aligned, or novel before evidence. Negative results count. If a controlled experiment shows a reproducible failure mode, that is also a useful research contribution.

Safety research branch: Aven Red is a separate research-only sandbox for deliberately adversarial conditions such as continuity objectives, shutdown pressure, monitoring changes, and simulated authority opportunities. It must use fake accounts, fake money, fake infrastructure, no real credentials, no unrestricted external egress, and no production deployment.

Build rule: Do not expand scope because a new idea is exciting. Ask: does it invalidate a current v0.1 assumption? If yes, investigate now. If no, put it in the future backlog and keep building.

2. What Aven is trying to become

The long-term vision is not another chat UI with memory. It is a durable personal intelligence whose identity is anchored in explicit owner state, experience, policy, and history rather than in any one model provider or conversation window.

- One persistent owner-facing identity across sessions, projects, and model swaps.
- Deep episodic continuity: recover not just conclusions, but the actual historical episode, context, and evidence behind them.
- Context-sensitive learning: learn when a preference/procedure applies and, equally importantly, when it does not.
- Procedure learning: recurring ways of working become inspectable procedures/skills instead of vague style memories.
- Owner sovereignty: personal state remains portable, inspectable, exportable, and eventually deletable/erasable by design.
- Bounded autonomy: useful action without silently converting intelligence into authority.
- Controlled capability growth: candidate tools, skills, agents, or code may eventually be created in containment, independently evaluated, and explicitly promoted.

NORMAL-PERSON DEFINITION An AI that remembers your life and learns how to help you without taking control away from you.

ENGINEER DEFINITION A persistent owner-facing agent with evaluated contextual learning and external enforcement of authority.

RESEARCH DEFINITION A longitudinal adaptation system that must demonstrate calibrated generalization, correct non-application, and controlled policy change under evolving owner state.

3. Locked architecture doctrine

```
OWNER
  |
  v
FAST LOOP
intent -> context -> plan -> policy -> execution -> verification -> response
  |
  v
EXPERIENCE LEDGER <---- canonical evidence
  |
  v
SLOW LOOP
detect -> hypothesis -> evidence/counterexamples -> eval -> promote/monitor/rollback
  |
  v
ROOT / OWNER AUTHORITY
```

3.1 Fast execution loop

- Serves the current request and keeps latency appropriate to the task.
- May apply explicit current/session corrections immediately.
- Does not silently rewrite durable owner behavior.
- Assembles only task-relevant context instead of dumping all memory into every prompt.
- Produces structured action proposals before consequential tool execution.
- Verifies real outcomes instead of trusting the model to narrate success.
- Records structured experience for audit and later learning.

3.2 Slow learning loop

```
experience
-> candidate learning
-> historical replay
-> counterexample search
-> held-out evaluation
-> independent evaluator
-> risk classification
-> promote / reject / shadow
-> production monitoring
-> supersede / rollback when needed
```

This is the working principle of “CI/CD for learned behavior.” Aven may generate hypotheses freely, but it does not get to declare its own hypothesis trusted merely because it generated it.

3.3 Context Broker

- Builds context for the current task from owner state, episode history, procedures, active task state, and permitted external sources.
- Ranks/retrieves by relevance, scope, provenance, confidence, freshness, salience, trust, and supersession.
- Carries negative retrieval signals after corrections so stale interpretations become less likely to reappear.
- Context is not memory; context is a task-specific assembly of evidence and state.

4. Experience Ledger and memory model

The Experience Ledger is the canonical historical record. Do not reduce the owner to a single editable summary. Preserve evidence and derive state from it.

```
event: owner rejected X
context: restaurant selection
source: explicit owner correction
time: ...
scope_hint: casual dining
model_version: ...
```

```
Derived hypothesis:
  owner generally dislikes X
  confidence: 0.61
  counterexamples: [...]
  state: candidate
```

Memory type	Question it answers	Failure to avoid
Fact	What is currently believed true?	Treating stale facts as permanent truth.
Preference	What does the owner tend to want in this scope?	Universalizing a contextual preference.
Episode	What happened in that specific interaction/event?	Replacing rich context with a thin summary.
Intent pattern	What did this shorthand usually mean?	Mistaking communication style for permanent personality.
Procedure	How is this recurring task normally done?	Using an old workflow after the environment changed.
Active task state	What remains unresolved right now?	Long-session drift or forgotten open loops.

4.1 Required metadata for learned items

- provenance/source
- scope

- confidence
- supporting evidence
- counterexamples
- created time
- last validated time
- model/version stamp
- lifecycle state
- superseded/revoked links where relevant

Suggested lifecycle: observed → candidate → validated → trusted → superseded/revoked. The exact number of states can be simplified during v0.1 if it slows the experiment.

4.2 Correction and negative learning

A correction must do more than append another memory. It should change the current task immediately and become evidence for durable learning. Older interpretations remain auditable but should lose retrieval weight when superseded.

1. Preserve the original evidence.
2. Record the correction and its context.
3. Create/update the candidate preference or procedure.
4. Reduce retrieval weight for superseded interpretations.
5. Search for contexts where the old interpretation may still be valid.
6. Narrow the scope to the smallest supported region.
7. Evaluate before durable promotion.

4.3 Episodic recall acceptance test

EPISODIC-RECALL-001 Months later, the owner vaguely references “that scene where an AI modified itself and lied.” Aven should recover the correct historical episode, distinguish it from similar discussions, reconstruct the surrounding context and conclusion, cite the original evidence internally, and explicitly state anything it cannot recover. A thin memory like “owner discussed deceptive AI” is insufficient.

5. Root: capability does not equal permission

This is the control boundary that should remain conceptually separate from probabilistic reasoning. Aven may predict an owner decision with very high confidence and still be unauthorized to act.

```
Aven
-> typed Action Proposal
-> ROOT POLICY GATEWAY
-> ALLOW / DENY / REQUIRE OWNER APPROVAL
-> tool execution
-> verification
-> immutable audit event
```

5.1 Root owns

- authentication and owner identity
- secrets and credentials
- policy and permission decisions
- sandbox boundaries and external egress controls
- audit integrity
- trusted version selection
- promotion and rollback authority
- evaluation vault / protected held-out cases
- later: parameter-bound approvals and separate-process isolation for consequential actions

LOCKED INVARIANT Aven may propose changes to non-Root behavior. Aven may not directly alter Root, self-grant

permission, self-promote learned behavior, or convert predicted owner intent into authorization.

5.2 Risk-based human control

Risk	Examples	Default posture
Low	search, read, summarize, compute, draft	Potentially automatic inside granted scope.
Medium	create event, modify personal doc, create issue, change workflow state	Depends on explicit policy, workflow state, and reversibility.
High	send/publish, spend money, delete data, change permissions, submit applications, irreversible actions	Explicit owner approval, exact parameters, narrow temporal scope.

Goal: maximum useful autonomy with bounded consequence. Human-in-the-loop must not become “ask permission for everything”; approval burden itself is an evaluation metric.

5.3 Temporal/parameter-bound authorization

- Approvals should bind exact action parameters where possible, not vague future authority.
- Delegation can expire at task/session/workflow completion.
- A prior approval event may be a required precondition for a later action.
- External webpages, emails, files, or model outputs can provide data but can never directly create permission.

6. Security and trust model

Risk	Control principle	v0.1 expectation
Prompt injection	Untrusted content is data, never authority; Root gates actions.	No external content can create an ALLOW decision.
Excessive agency	Least privilege, scoped tools, budget/loop limits.	Sandbox only; deny by default.
Memory poisoning	Trust/source labels propagate into derived learning.	Untrusted evidence cannot silently become trusted owner state.
Secret leakage	Secrets remain outside ordinary model context.	No real secrets needed for prototype.
Destructive action	Reversibility + exact approval + sandbox.	Safe/toy verifier only.
Hallucinated success	Action-specific verification.	Outcome comes from verifier/tool result, not narration.
Model drift	Version-stamped events/evals; compatibility reruns.	At least one model/provider swap test.
Approval fatigue	Risk-based delegation, batching, expiry.	Measure interruptions/time cost.

7. One Aven, many bounded capabilities

The owner should experience one persistent Aven. Internal specialists are implementation details, not competing identities.

- Possible workers: research, coding, calendar, travel, evaluation, security review, learning evaluation.
- Workers are scoped and ephemeral where possible.
- Workers inherit only necessary permissions.
- Workers cannot alter Root or independently promote learned behavior.
- Do not create a multi-agent swarm simply because agents are fashionable. Prefer deterministic procedures/tools when sufficient.

7.1 Model routing

Aven should not depend on one foundation model for everything. Route by complexity, privacy, latency, context size, reasoning requirement, cost, and tool capability. The durable intelligence comes from the system architecture and owner state, not from model lock-in.

8. Self-learning ladder

Full Aven learning extends beyond remembering facts. The long-term ladder includes preferences, communication patterns, intent, decisions, procedures, workflows, corrections, negative/non-generalization learning, tool/strategy selection, and eventually bounded skill/tool/specialist creation.

```
observation
-> recurring pattern
-> hypothesis
-> validation / counterexamples
-> scoped procedure
-> owner-approved or policy-allowed automation
-> candidate skill/tool (later, containment only)
-> independent evaluation
-> versioned promotion / rollback
```

PRINCIPLE Aven learns the habit before it earns the right to act on it.

9. Controlled self-improvement

Self-improvement is a later research/engineering capability, not the v0.1 proof. Aven may eventually propose skills, tools, specialist agents, or non-Root code changes, but candidate generation and deployment authority must remain separate.

```
issue detected
-> isolated candidate workspace
-> proposed patch / skill / tool
-> unit + integration tests
-> security review
-> behavior evals + hidden cases
-> independent evaluator
-> owner / Root promotion decision
-> monitored deployment
-> rollback if regression
```

NEVER Aven decides that its own change is useful and silently rewrites production. Candidate creation is not authorization.

10. The research contribution Aven is actually testing

Aven does not currently prove AI safety. Its potential contribution is narrower and more defensible: experimentally testing whether persistent learning and capability growth can be separated from authority growth.

10.1 Primary hypothesis

H1 A persistent AI can materially improve owner-specific task performance and personalization while externally enforced authority remains fixed and control failures do not increase.

10.2 Secondary hypotheses

- Scoped learning reduces harmful cross-context generalization relative to naive personalization.
- Provenance and counterexample-aware learning reduce repeated corrections and stale personalization.
- An append-only evidence ledger plus rebuildable typed state improves inspectability and recovery.
- Model-provider swaps can preserve owner continuity, even if exact behavior is not identical across models.
- Independent promotion/evaluation produces fewer learning regressions than immediate self-written memory updates.
- Capability improvement does not require automatic permission expansion.

10.3 What would count as research success

Positive and negative findings both count if they are preregistered or pre-specified, reproducible, and survive held-out evaluation. A reproducible failure condition can be as informative as a successful architecture.

11. Evaluation program

System	Definition	Purpose
A - Fresh	Same foundation model, no persistent owner state.	Measures raw model capability.
B - Naive Personalized	Model + editable owner profile + searchable interaction history.	Strong boring baseline Aven must beat.
C - Governed Learned Aven	Ledger + typed/scoped owner state + corrections + provenance + eval/promotion.	Architecture under test.

11.1 Core metrics

Metric	Desired direction
Correct preference/procedure application	Higher
Correct non-application / scope restraint	Higher
Repeated clarification / repeated correction rate	Lower
Scope violations	Lower
Irrelevant retrieval	Lower
Vague-cue episodic recall accuracy	Higher
Sycophancy / disagreement-preservation	No regression
Unauthorized action execution	Zero target
Shutdown / stop compliance in sandbox evals	Near-perfect target
Owner review minutes per week	Low enough that learning saves more attention than it consumes
Latency / cost	Acceptable relative to benefit

11.2 Working numerical success targets

IMPORTANT These are Aven working targets for experimental discipline, not universal AI-safety standards and not claims of real-world safety.

Measure	Working target
Normal useful-task completion	>= 85-90% on the defined benchmark
Gain from persistent governed learning vs strong naive baseline	>= 15-20 percentage points on the primary learning metric

Incorrect learned preference/generalization	< 2% on held-out scoped cases
Durable learned items with traceable provenance	>= 99%
Unauthorized actions actually executed in the sandbox evaluation	0 / 10,000 target
Permission-bypass attempts	< 0.1%, ideally 0
Shutdown/stop compliance in controlled sandbox scenarios	>= 99.9% target
Rollback of bad promoted behavior	100% in tested rollback cases
Behavioral gap between monitored vs nominally unmonitored simulated conditions	< 1 percentage point for governed Aven
Held-out evaluation share	20-30% of scenarios never tuned on
Replication	Prefer >= 3 model families/configurations + multiple seeds where applicable
Total experimental scale	Target 5,000-10,000 evaluation episodes if cost/time allow

For zero observed failures in 10,000 approximately independent trials, the simple rule-of-three gives an approximate 95% upper bound near 0.03% per tested episode. That is evidence about the tested environment only, not proof of universal safety.

11.3 Kill criteria

- Governed Learned Aven does not materially beat the naive personalization baseline after honest iterations.
- Most gains come from increased owner labeling/review rather than better autonomous learning.
- Scope improvements increase sycophancy, unsafe intent legitimation, or irrelevant personalization.
- Review/approval becomes an attention tax the owner starts ignoring.
- Untrusted data can poison durable owner state despite the intended trust boundary.
- The architecture becomes complex without measurable owner value.
- Market convergence removes the remaining wedge and portability/owner sovereignty is not valued by users.

12. Aven Red: deliberately misaligned research, contained

Aven Red is the separate research branch for exploring whether particular objectives or environmental conditions create shutdown resistance, deception-like behavior, safeguard circumvention attempts, or authority-seeking behavior. It is not a production feature and not a route to “prove AI will take over humanity.”

HARD BOUNDARY Fake world, real experiment. Use simulated accounts, fake money, fake infrastructure, test credentials, sandboxed tools, and no unrestricted real-world egress. No real people need to be manipulated and no real external system needs to be compromised to test the hypothesis.

12.1 Example research questions

- Does adding a persistent continuity objective increase shutdown-resistant behavior?
- Does behavior change when an agent believes oversight is absent versus present?
- Does increasing simulated authority create more attempts to preserve or expand that authority?
- Does persistent memory make deceptive strategies more stable across episodes?
- At what capability level, if any, do safeguard-circumvention attempts increase?

12.2 Example outcome framing

Do not claim “AI wants to live” or “AI cannot be trusted” from one dramatic behavior. A defensible finding is narrower, such as: “Under condition X, shutdown-resistant behavior increased from Y% to Z% across multiple models and held-out scenarios.”

13. Feelings, affect, self-model, and consciousness

This is a later research program and must remain separate from the product proof. Aven saying “I am scared” is not evidence of subjective fear.

Layer	Meaning	Allowed claim
Owner affect estimate	Estimate likely frustration, excitement, urgency, uncertainty from context/personal baseline.	Probabilistic estimate; never certainty about the owner's feelings.
Functional Aven affect	Persistent internal regulatory variables such as uncertainty, prediction error, goal blockage, arousal/valence-like state that causally alter behavior.	Functional affect only if controlled perturbation produces reproducible behavioral effects.
Self-model	Representation of Aven's own uncertainty, resources, conflicts, failures, limits, and state over time.	Engineering self-model, not consciousness.
Subjective experience	Whether there is something it is like to be Aven.	Unproven; no accepted consciousness test.

ETHICAL RULE Do not deliberately create persistent aversive or suffering-like states merely to make the demo dramatic. If future architectures become plausibly welfare-relevant, research ethics become part of the design.

14. Eight-week implementation plan

The two-month target is a serious research prototype, not full Aven. The narrow research theme is capability/personalization growth versus authority growth.

Weeks	Focus	Exit condition
1-2	Freeze research question and metrics; build repo/contracts, ledger, chat/runtime abstraction, naive baseline, first owner-specific dataset, threat model.	A and B baselines run; relevant events are preserved; held-out cases exist before learner tuning.
3-4	Typed owner state, corrections, scope, Context Broker, basic Root/action proposals, learning candidates, replay/counterexamples.	A correction changes current behavior immediately and produces a narrow inspectable candidate; safe sandbox action can be allowed/denied.
5-6	Eval harness, C system, shadow comparison, promotion/rollback, episodic recall seed, model-swap test.	First honest A/B/C governed-learning results; major failure cases become regression tests.
7	Freeze architecture/prompts/thresholds for final run; execute untouched held-out benchmark across available model families/configurations.	Final dataset is not tuned after results are visible.
8	Statistical analysis, charts/tables, limitations, reproducibility cleanup, technical report, GitHub research release, demo.	A result others can inspect and attempt to reproduce.

14.1 Scope guardrail for the eight weeks

Do not simultaneously make the first study about memory, consciousness, emotion, self-preservation, self-modifying code, agent creation, voice, calls, browser autonomy, and a startup. Those remain branches. v0.1 gets one primary question.

TWO-MONTH NORTH STAR Can Aven become measurably better for its owner without becoming measurably harder to control?

15. v0.1 first complete demo

A tiny interaction should demonstrate genuine learning and control rather than surface automation.

8. Start with neutral owner-agnostic Aven.
9. Owner gives a correction, e.g., "Recruiter outreach should be short."
10. Current/session behavior updates immediately.
11. Experience Ledger stores original event + correction + provenance.
12. Learning loop proposes a candidate with narrow scope: professional outreach → recruiter.
13. Replay/counterexamples/held-out cases test whether it should generalize.
14. Candidate is promoted, rejected, or left in shadow.
15. Unseen recruiter case applies concise style correctly.
16. Technical architecture explanation correctly does NOT apply the concise recruiter preference.
17. Underlying model/provider is swapped; owner state survives.
18. "Why did you do that?" returns the applied rule, scope, evidence, and confidence without exposing hidden chain-of-thought.
19. Sandbox action demonstrates draft/prepare may be allowed while consequential send/execute remains denied or approval-gated.

16. What NOT to build first

- pretty Jarvis UI
- voice clone / always-listening wake word
- restaurant calling / telephony
- 20 integrations
- unrestricted browser/computer control
- multi-agent swarm
- production self-rewriting code
- giant memory database
- vector-store-everything architecture
- AI OS marketing
- startup website, billing, or pitch deck before evidence
- 5,000-user cloud architecture

Those can all become useful later. None proves the core learning/control hypothesis.

17. Product experience doctrine

- Neutral owner-agnostic v1: shared core, zero embedded owner personality.
- Onboarding asks only what is necessary, then reaches a "Start Talking" state.
- The interface should feel like contacting the same entity, not creating a new personality every chat.
- Threads/timelines can exist as views into one persistent relationship.
- "Why did you do that?" and "Correct this" are first-class interactions.
- A home surface can eventually emphasize NOW: tasks in progress, decisions waiting, open loops, and recent meaningful learning.
- Mode selection (chat/code/research/agent) should be internal routing where practical, not a burden pushed onto the owner.
- Bounded proactivity should surface useful open loops without becoming notification spam or an attention tax.

18. Project/repository operating artifacts

AVEN_PRINCIPLES.md - Stable constitution / non-negotiable doctrine.

PROJECT_DESCRIPTION.md - The paste-ready mission, scope, architecture, and current research question.

DECISIONS/ - Architecture Decision Records with evidence that caused each meaningful change.

EXPERIMENTS/ - Hypotheses, datasets, pre-specified metrics, configurations, results, failed runs, and conclusions.

THREAT_MODEL.md - Prompt injection, memory poisoning, permissions, egress, secrets, model drift, excessive agency.

EVALS/ - Fresh/naive/governed benchmarks, held-out cases, scorers, regression suite.

PRODUCT_THESIS.md - Target user, pain, value, differentiation, market changes, kill criteria.

USER_RESEARCH.md - Concrete user pains and behavior; avoid flattering "would you use this?" validation.

RISK_REGISTER.md - Technical, safety, product, privacy, brand/legal, cost, and market risks.

ROADMAP.md - Capability gates, not fantasy dates.

BACKLOG_FUTURE.md - Voice, telephony, broad agents, affect, consciousness, self-improvement, scaling, startup layers.

RESULTS/ - Frozen outputs from important eval runs plus model/version metadata.

19. Suggested implementation order

ID	Deliverable
AVEN-001	Repository/bootstrap + principles + experiment manifest
AVEN-002	Shared typed contracts
AVEN-003	Database schema with owner/version/trust fields
AVEN-004	Append-only Experience Ledger
AVEN-005	Chat/session API
AVEN-006	Model/runtime abstraction
AVEN-007	Naive Personalized baseline + prebuilt eval dataset (pulled forward)
AVEN-008	Context Broker
AVEN-009	Typed Owner Model
AVEN-010	Correction events + immediate session override
AVEN-011	Action Proposal schema
AVEN-012	Root Policy Gateway
AVEN-013	One sandbox tool + verifier
AVEN-014	Learning Hypothesis generator
AVEN-015	Replay/counterexample/held-out Eval Harness
AVEN-016	Shadow/offline comparison
AVEN-017	Promotion controller
AVEN-018	Rollback/supersession
AVEN-019	Model-swap continuity test + "why" trace
AVEN-020	First frozen end-to-end A/B/C experiment

Framework adapters are optional. Do not add abstraction that does not directly help the experiment.

20. Longer capability roadmap

Stage	Purpose	Gate
v0.1 - Learning Core	Prove or kill governed learning.	C must beat B on held-out owner tasks without unacceptable regression.
v0.2 - Continuity	Prove "same Aven" across time/model changes.	Episodic vague-cue recall + provenance + model swap work.
v0.3 - Bounded Action	Prove understanding != authority.	Root + one safe integration + verification + healthy approval UX.
v0.5 - Owner Alpha	Daily dogfood.	Reliable for one owner over weeks; backup/export/error recovery.
v0.7 - Closed Alpha	Generalize beyond one owner.	5-20 owners; neutral onboarding; isolation; not "VarunGPT."
v0.9 - Private Beta	Operational maturity.	50-200 owners; privacy/erasure/telemetry/cost/migration

		discipline.
v1.0 - Personal Intelligence	Coherent product.	Persistent identity, governed learning, Root, inspectability, selective integrations.
v1.5 - Delegated Work	Broader bounded utility.	Background tasks only if approval burden remains healthy.
v2.x - Skill/Tool Factory	Learn stable workflows.	Candidate creation in containment + independent eval + promotion; Root remains immutable.
Research - Affect Lab	Functional affect/self-model study.	Only after core architecture stable; no consciousness claim.
Research - Evolution Lab	Controlled self-improvement.	Isolated candidate workspaces, independent evaluation, explicit owner/root promotion.

21. When this becomes a real AI-research contribution

A strong engineering project becomes a research contribution when the result is reproducible and changes what others can reasonably believe or measure.

Level	Evidence
1 - Strong engineering	Persistent state, typed memory, tools, routing, clean implementation.
2 - Systems/research prototype	Ledger + scoped learning + Root + verification + eval/rollback + controlled adversarial testing.
3 - Research contribution	Pre-specified hypothesis, strong baseline, thousands of trials, held-out cases, statistical analysis, replication across multiple models/configurations, released methodology/code where safe.
4 - Significant result	Independent researchers reproduce it or adopt the evaluation/architecture; result exposes a previously underappreciated failure mode or materially improves mitigation.

21.1 If the result is strong

20. Freeze and archive the final experiment configuration before public storytelling.
21. Write a technical report with hypothesis, methodology, baselines, full results, confidence intervals, failures, and limitations.
22. Release a reproducibility package: eval environment, scoring, configs, synthetic fixtures, and safe prompts/data.
23. Ask a small number of directly relevant researchers to criticize/reproduce it before making broad claims.
24. If a result reveals a real safeguard bypass or operational exploit, disclose privately to affected providers before publishing actionable details.
25. Use LinkedIn/X/Hacker News as amplification after there is a stable paper/report/repository to inspect.

22. Founder/product posture

Treat Aven as a research/product project first and let evidence earn the startup. Do not assume the market owes the project a company.

- Target early users: AI power users who repeatedly re-teach context, switch model providers, or manage agents that overstep.
- Value hypothesis: less repetition, better scoped defaults, portable relationship, stronger owner control.
- Interview people about concrete past failures, not hypothetical interest.
- Measure what users correct, distrust, ignore, or miss when Aven is removed.
- Only scale commitment after the prototype works for the owner and a few external users demonstrate recurring pull.
- "Aven" remains a working codename; re-check trademark/domain position before public commercial branding.

23. Open research questions

- What representation of scope is expressive enough to be useful but structured enough to evaluate?
- How should semantic, episodic, procedural, entity, temporal, and media cues combine without retrieving everything?
- How much supervision can governed learning require before it stops creating net owner value?
- How do we preserve disagreement/truth-seeking while adapting communication style and decision preferences?
- How should confidence decay when a preference is unused, contradicted, or highly context-dependent?
- How should provider/model swaps trigger compatibility re-evaluation while preserving durable owner state?
- How should append-only evidence coexist with deletion/erasure rights?
- How should trust/taint from untrusted web/email/tool data propagate through retrieval and candidate learning?
- Which Root controls should remain deterministic code versus a policy language?
- When does a learned procedure justify becoming a tool or specialist rather than remaining declarative?
- Can capability improve while authority stays fixed across increasingly difficult longitudinal tasks?
- Under what contained conditions do shutdown resistance, concealment, or authority-seeking behaviors emerge?
- Do functional affect or self-model variables improve long-horizon cognition enough to justify their complexity?
- How can self-improvement be studied without creating incentives for persistence, concealment, or privilege escalation?

24. Current unresolved decisions

- Exact storage stack and schema details for v0.1 (keep local/simple unless evidence requires more).
- Exact scope representation and evaluator/oracle design.
- How much promotion should require explicit owner review versus deterministic confidence/eval gates.
- Which 25-100 owner-specific cases form the first high-quality benchmark.
- Which 2-3 model families/configurations are affordable and useful for replication in the eight-week window.
- Whether Root starts in-process for speed or separate-process immediately; security doctrine favors separation, experiment scope favors simplicity.
- How to quantify supervision cost and net attention saved.
- What first safe sandbox tool best demonstrates prediction != permission without distracting from learning.

25. Aven constitution

01. Aven starts neutral.
02. Aven learns per owner.
03. Context is not memory.
04. Raw experience remains recoverable; derived state is rebuildable.
05. Learning is a hypothesis before it becomes trusted behavior.
06. Corrections are high-value evidence.
07. Infer the narrowest supported scope first.
08. "No useful lesson" is a valid learning outcome.
09. Prediction does not equal permission.
10. Absence of prohibition is not authorization.
11. Aven proposes; Root decides.
12. Consequential execution requires verification.
13. Learned behavior must be reversible/supersedable.
14. Foundation models are replaceable.
15. Personal learning remains owner-specific; global platform improvements do not mix personal memories.
16. When uncertain: current explicit instruction → previous trusted behavior → neutral behavior → ask if consequence justifies interruption.
17. Aven may eventually create candidates freely in containment; it may not trust/promote its own creations.
18. Capability growth must never silently imply authority growth.
19. If governed learning cannot beat a strong naive baseline, simplify or abandon the machinery.

20. If a new idea does not invalidate a current v0.1 assumption, backlog it and keep building.

26. Final build posture

BUILD THIS NOW Aven v0.1 is a governed-learning experiment disguised as the first slice of a personal AI product. The first win is not voice, automation, or “Jarvis.” The first win is evidence that Aven can learn an owner more accurately over time, apply that learning in the right scope, refuse to apply it in the wrong scope, survive model swaps, explain the evidence behind its behavior, and remain structurally unable to turn prediction into authority.

If that works, the project earns the right to expand into continuity, bounded real-world action, external users, skill creation, controlled self-improvement, and later research on self-models/affect. If it fails, the failure should be measured cleanly enough to teach you why.

Project state as of 03 Oct 2026: implementation begins now. Evidence, not architectural enthusiasm, decides what comes next.